

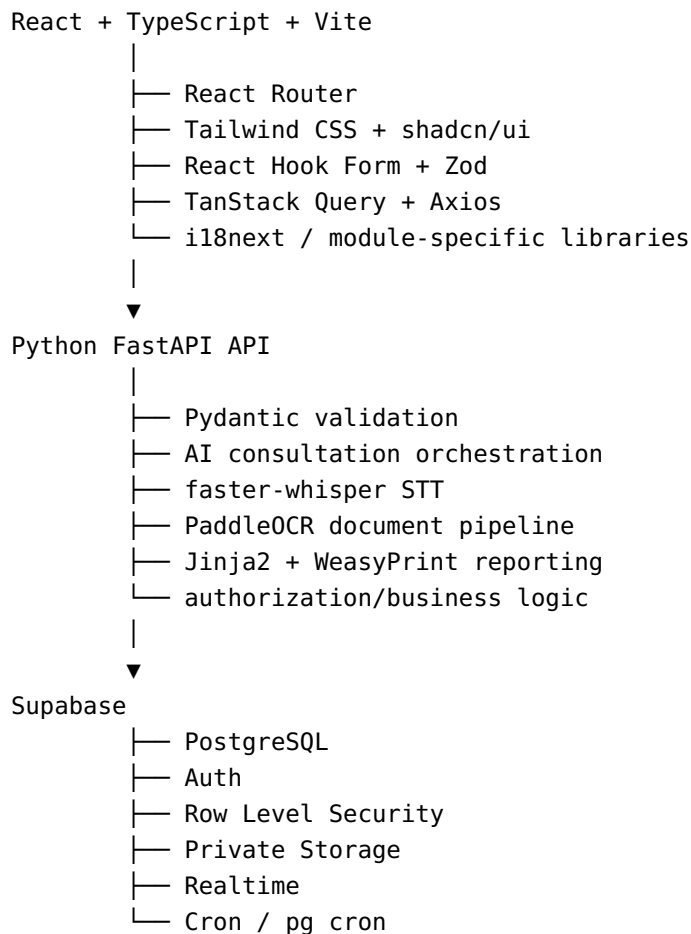
TOOLS.md — Patient Case-Taking Software

Executive summary

This document defines the recommended implementation stack for the **SIH Patient Case-Taking Software** and maps the technology choices to the project's **nine approved modules**. The recommendations preserve the functional architecture in the approved project documentation: patient registration, voice-led AI case-taking, optional previous-document digitization, patient review/edit/confirmation, physician-ready PDF generation, report history, notifications, profile/settings, Hindi-English support, help/support, hospital services, and a separate authenticated hospital admin portal. Most importantly, the software remains a **case-taking and information-structuring system, not an autonomous diagnostic system**.

filecite turn0file

The recommended baseline architecture is:



React remains an appropriate component-oriented UI layer, while Vite provides the development/build toolchain; TypeScript should be used instead of plain JavaScript so that frontend models, API contracts, and module interfaces receive static type checking. ¹ FastAPI is recommended for the API and AI/document-processing backend because it is Python-native, integrates Pydantic validation and OpenAPI generation, and fits the Python ecosystem used by faster-whisper, PaddleOCR, OpenCV, PyMuPDF, and report generation. ²

Supabase should be treated as the project's primary **data platform**, rather than introducing MySQL alongside it. It provides PostgreSQL, authentication, storage, realtime capabilities and related platform services, while Supabase Auth JWTs integrate with PostgreSQL Row Level Security. ³ For sensitive patient data, RLS must be enabled on exposed tables, privileged keys must remain server-side, uploaded medical documents must remain in private storage, and time-limited signed URLs should be used when a temporary document link is necessary. ⁴

The architecture intentionally keeps external AI, OCR, speech, email, payment, map-tile, and push-notification vendors replaceable. Where the functional documentation does not specify a provider, this file labels that dependency **TBD / provider unspecified** rather than inventing a cloud dependency. The project documentation itself explicitly deferred technology selection until after the functional architecture was finalized. `filecite@turn0file0`

Recommended core stack

Layer	Primary choice	Primary responsibility	Main alternative	Decision
Frontend language	TypeScript	Typed React code and API models	JavaScript	Use TypeScript ⁵
Frontend	React	Patient and admin interfaces	Vue / Angular	Use React ⁶
Build tooling	Vite	Development server and production build	Next.js / another bundler	Use Vite SPA architecture ⁷
Routing	React Router	Module/page routing	TanStack Router	Use React Router ⁸
Styling	Tailwind CSS	Responsive UI styling	CSS Modules	Use Tailwind ⁹
UI components	shadcn/ui	Accessible reusable UI primitives	Material UI	Use shadcn/ui ¹⁰
Forms	React Hook Form	Form state	Formik	Use RHF ¹¹
Client validation	Zod	TypeScript schemas	Valibot / Yup	Use Zod ¹²

Layer	Primary choice	Primary responsibility	Main alternative	Decision
Server-state	TanStack Query	API cache and mutations	SWR	Use TanStack Query ¹³
HTTP	Axios	Frontend → FastAPI requests	Native <code>fetch</code>	Use Axios consistently ¹⁴
Backend	Python + FastAPI	APIs, business logic, AI/OCR	Django/DRF / Node	Use FastAPI ²
API schemas	Pydantic	Request/output validation	dataclasses/ manual validation	Use Pydantic ¹⁵
Database	Supabase PostgreSQL	Relational system of record	Managed PostgreSQL	Use Supabase PostgreSQL ¹⁶
Authentication	Supabase Auth	Patient/admin identity	Keycloak / Firebase Auth	Use Supabase Auth ¹⁷
Data authorization	PostgreSQL RLS + FastAPI checks	Patient ownership and authorization	Backend-only ACL	Use both layers ¹⁸
Files	Supabase Storage	Private health documents	S3-compatible storage	Use private buckets ¹⁹
Realtime	Supabase Realtime	In-app notification updates	WebSockets + Redis	Use Realtime ²⁰
Scheduled work	Supabase Cron / pg_cron	Reminder scans and recurring jobs	APScheduler / Celery	Use DB-backed Cron ²¹
Speech-to-text	faster-whisper	English/Hindi transcription	Managed STT	Use local/self-hostable baseline ²²
OCR	PaddleOCR	Medical document OCR	Tesseract / managed OCR	Use PaddleOCR baseline ²³
PDF extraction	PyMuPDF	PDF parsing/ rendering	pypdf	Use PyMuPDF ²⁴
Image processing	OpenCV + Pillow	OCR preprocessing	ImageMagick	Use both selectively ²⁵
Report templates	Jinja2	HTML report templates	Manual string templates	Use Jinja2 ²⁶
PDF generation	WeasyPrint	HTML/CSS → PDF	ReportLab / Chromium	Use WeasyPrint ²⁷

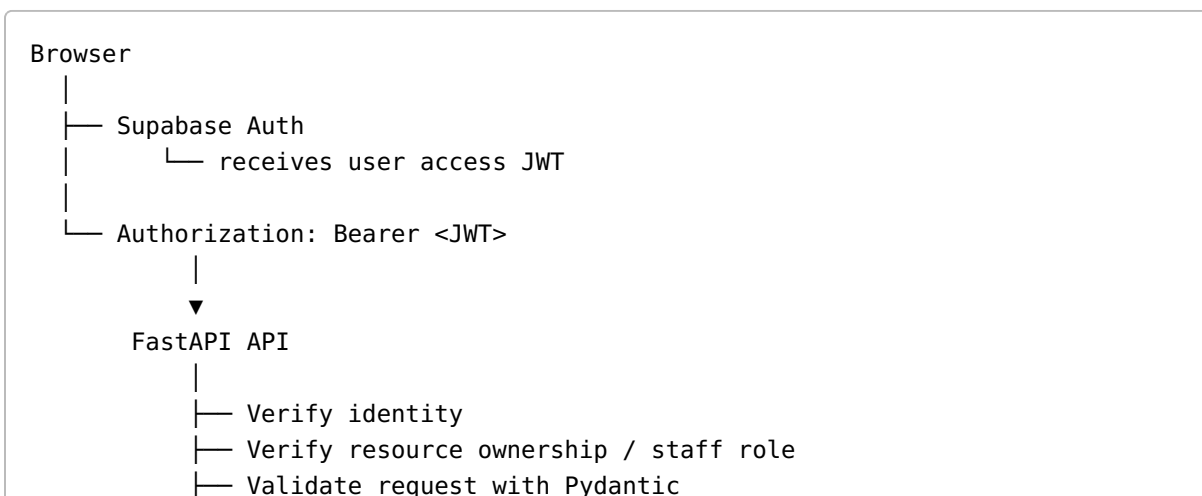
Layer	Primary choice	Primary responsibility	Main alternative	Decision
Translation	i18next + react-i18next	English/Hindi UI	FormatJS	Use i18next ²⁸
Maps	Leaflet	Hospital/location maps	MapLibre	Use Leaflet ²⁹
Map data/tiles	OpenStreetMap-compatible source	Map display	Commercial tile service	Demo: OSM; production provider TBD ³⁰
CI/CD	GitHub Actions	Test/build/deploy pipelines	GitLab CI	Use GitHub Actions ³¹
Containers	Docker	Reproducible backend runtime	Podman	Use Docker/OCI ³²

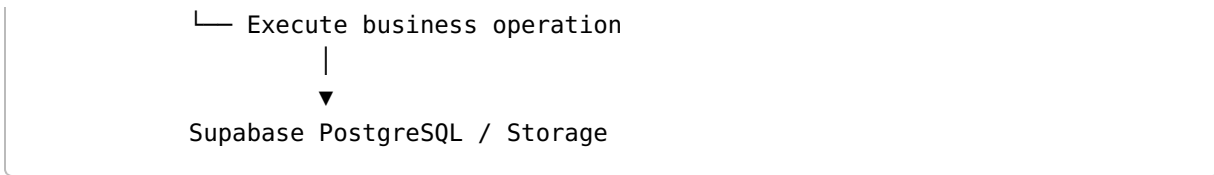
Project-wide architecture and tool decisions

The frontend should not become the trusted location for medical business logic. Patient input begins in React, but validation, authorization, AI orchestration, document processing, report finalization, appointment concurrency, admin actions, and privileged Storage operations belong in FastAPI and/or PostgreSQL. Client-side Zod validation improves usability but never replaces Pydantic/backend/database validation. FastAPI uses Pydantic models for typed validation and automatically incorporates those models into its API schema/documentation. ¹⁵

The frontend may directly use `@supabase/supabase-js` for authentication session management and authenticated Realtime subscriptions, but privileged service/secret credentials must never be shipped in a Vite bundle. Supabase explicitly distinguishes publishable/client credentials from privileged server-side credentials, and its RLS guidance notes that privileged service access can bypass RLS. ³³

A recommended request flow is:





Supabase JWTs carry identity/authorization information and can be verified using the project's signing-key infrastructure; the official documentation recommends using supported verification mechanisms rather than implementing signature algorithms manually. ³⁴

Primary tool decision register

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
TypeScript	Frontend across Modules 1–9	Enable strict type checking; share generated/API DTO types where practical	JavaScript	Pro: catches type errors earlier. Con: adds type definitions/build checks	No metered cloud cost; never embed backend secrets in typed frontend config. ⁵
React	All patient/admin interfaces	Use feature/module components rather than one monolithic dashboard	Vue, Angular	Pro: reusable component model. Con: architecture/state patterns remain team responsibility	Client-side code is public to users; never rely on React code to enforce authorization. ⁶
Vite	All frontend build/dev	Keep <code>VITE_*</code> variables limited to intentionally public configuration	Next.js, another bundler	Pro: focused SPA dev/build tooling. Con: no server-side framework features by itself	Anything injected into frontend output should be treated as public. ⁷
React Router	Module/page navigation	Protect routes for authenticated areas, but repeat access checks server-side	TanStack Router	Pro: clear module routing. Con: route guards are not security controls	A hidden admin route is not authorization. ⁸

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
Tailwind CSS	Modules 1–9	Establish design tokens, focus states, responsive breakpoints, readable font scale	CSS Modules, Material UI styling	Pro: consistent UI utilities. Con: class-heavy JSX if unmanaged	No patient data should ever be encoded into generated CSS/ classes. ⁹
shadcn/ui	Modules 1–9	Keep components in repository and customize accessibility/ healthcare UI	Material UI, Chakra UI	Pro: editable component source. Con: team owns component updates after adoption	Review components during dependency/ security upgrades. ¹⁰
Lucide React	Modules 1–9	Use icon + text label for critical actions; do not use icon-only emergency controls	Heroicons	Pro: consistent iconography. Con: icons alone may be ambiguous	No external runtime service cost
React Hook Form	Modules 1, 5, 7, 8, 9	Pair with Zod resolver; display accessible field errors	Formik	Pro: efficient form-state model. Con: schema coordination still required	Do not log complete medical form state to browser analytics. ¹¹
Zod	Frontend validation in Modules 1, 5, 7, 8, 9	Use schemas for email/mobile/range/ enum checks; backend validates again	Valibot, Yup	Pro: TypeScript-oriented schemas. Con: duplicate validation with backend is intentional	Client validation is UX, not a security boundary. ¹²
TanStack Query	Server-state across Modules 1–9	Centralize query keys; invalidate after mutations; avoid caching sensitive responses longer than needed	SWR	Pro: caching/ refetch/ mutation management. Con: cache lifecycle requires discipline	Sensitive records remain in browser memory during session; clear relevant caches on logout. ¹³

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
Axios	Browser → FastAPI	One API instance; attach bearer token; timeout; common error mapping	Native <code>fetch</code>	Pro: interceptors/ convenient request API. Con: additional dependency	Never print authorization headers or patient payloads in production logs. ¹⁴
FastAPI	Backend for Modules 1–9	<code>/api/v1</code> ; dependency-based auth; OpenAPI; async where I/O-bound	Django REST Framework, Node/NestJS	Pro: Python AI ecosystem and typed APIs. Con: CPU-heavy OCR/STT must not block request workers	Deploy only over HTTPS; segregate long AI/document jobs when load grows. ³⁵
Pydantic	Backend schemas and AI structured output	Separate input, internal and response models; reject unexpected high-risk fields where suitable	dataclasses + validators	Pro: structured validation. Con: valid structure does not guarantee clinical correctness	Validate LLM output before persisting it. ¹⁵
Supabase PostgreSQL	System of record for Modules 1–9	UUID keys, foreign keys, constraints, indexes, timestamps; migrations in Git	Managed PostgreSQL elsewhere	Pro: mature relational model. Con: schema/RLS design requires care	Usage/storage/ egress/ compute become paid at production scale; confirm current pricing before deployment. ³⁶
Supabase Auth	Account access, Modules 1, 5, 9 and all protected routes	JWT sessions; email verification as applicable; require stronger controls/ MFA for admins	Keycloak, Firebase/ Auth0-style provider	Pro: integrated with RLS. Con: external identity dependency when using hosted platform	Protect project/ admin accounts with MFA; healthcare production requirements require institutional review. ³⁷

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
PostgreSQL RLS	Patient isolation across all medical/operational tables	Policies such as <code>auth.uid() = patient_user_id</code> ; revoke unnecessary grants; test allow + deny cases	Backend ACL only	Pro: defense in depth. Con: incorrect policies can expose or block data	Mandatory for exposed patient tables; secret/service roles bypass normal patient protections and must stay server-side. ¹⁸
Supabase Storage	Modules 2, 3, 5, 7, 8, 9	Private buckets; store paths/metadata in DB; signed URLs only when necessary	S3-compatible object storage	Pro: integrated access. Con: object lifecycle is separate from DB rows	Never make prescriptions/reports public. Signed URLs should be short-lived because they remain valid until expiration. ¹⁹
Supabase Realtime	Modules 4, 8, 9	Private user channels; prefer controlled Broadcast architecture for app notifications	FastAPI WebSocket + Redis	Pro: low-friction realtime. Con: channel/policy design required	Never broadcast one patient's event on an unrestricted shared channel. ²⁰
Supabase Cron / pg_cron	Medicine reminders, housekeeping, notification scheduling	Use a small number of recurring workers that query due rows, rather than creating one cron job per reminder	APScheduler, Celery + Redis	Pro: durable schedule in DB. Con: unsuitable for long CPU-heavy AI processing	Cron should store minimal execution metadata; do not place PHI in job names. Supabase documents job execution limits and monitoring. ²¹

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
Supabase CLI	Database schema, RLS, seed and local development	Commit <code>supabase/ migrations</code> ; test with local stack; <code>db push</code> through controlled deployment	Alembic	Pro: migrations aligned with Supabase. Con: one more local tool/ runtime	Local Supabase is for development, not an externally exposed production service. ³⁸
supabase-js	Auth + Realtime in frontend	Browser uses only publishable key; use RLS	Custom REST clients	Pro: official Supabase JS SDK. Con: careless direct Data API usage can broaden attack surface	Explicitly expose only required tables/ functions and apply RLS first. ³⁹
supabase-py	FastAPI ↔ Supabase	Use server configuration; preserve user scope for patient operations where possible; privileged credentials only for controlled background/admin paths	Direct PostgreSQL driver	Pro: official Python client interface. Con: transactions may be easier in SQL/RPC for complex workflows	Protect privileged credentials as server secrets. ⁴⁰
MediaDevices / getUserMedia	Module 2 voice capture	Ask microphone permission only when consultation starts; require secure context; visible recording state	Native mobile capture	Pro: built into browser. Con: permission/ device variability	Audio is sensitive health data; avoid retaining raw audio unless functionally required and consented. ⁴¹
faster-whisper	Modules 2 and 6	Backend transcription; configure model/ device/compute type and explicit language when known	Managed STT, original Whisper	Pro: self-hosting/CPU-GPU flexibility. Con: compute and model-management burden	Local inference can reduce disclosure to external speech vendors; GPU hosting has infrastructure cost. ²²

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
Web Speech synthesis	Modules 2 and 6 voice output	Set utterance <code>lang</code> ; select Hindi/English voice where available; provide text fallback	Managed TTS	Pro: no backend TTS required for prototype. Con: voice availability/quality differs by browser/OS and voices can be local or remote	Do not assume identical privacy behavior on every browser/platform; production TTS provider remains replaceable. 42
LLM provider adapter — provider TBD	Module 2 dynamic questioning and report structuring	Define <code>LLMProvider</code> interface; require JSON/structured output; temperature low; clinical-safety system prompt; timeout/retry; no diagnosis	Self-hosted LLM server; managed LLM API	Pro: vendor independence. Con: team must maintain prompts/evaluation/provider adapter	Do not send identifiable health data to an external model until data processing, retention, residency, contractual and hospital requirements are approved.
PaddleOCR + PaddlePaddle	Module 2 uploaded document OCR	Select language/model deliberately; preprocess pages; retain confidence and source page references	Tesseract; managed document AI	Pro: local multilingual OCR stack. Con: handwriting/complex prescriptions remain error-prone and require verification	Local execution avoids third-party OCR transfer but consumes CPU/GPU. Current PaddleOCR documentation includes Hindi support through its language/model pipeline; do not assume every multilingual model covers every language. 23

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
PyMuPDF	Modules 2 and 3 backend document processing	Extract existing text first; render only pages requiring OCR	pypdf, pdfplumber	Pro: fast PDF processing. Con: document edge cases require testing	Do not execute embedded content; treat uploads as untrusted. ²⁴
OpenCV headless	Module 2 OCR preprocessing	Deskew, grayscale, threshold/denoise only when benchmarks show improvement	scikit-image	Pro: strong image pipeline. Con: aggressive preprocessing can destroy text	Keep original upload unchanged; OCR derivative should be reproducible. ⁴³
Pillow	Modules 2, 5, 7, 9 image handling	Decode/resize images; enforce dimension limits	OpenCV alone	Pro: simple image processing. Con: malformed images remain an attack surface	Keep Pillow current and reject unreasonable decompression/ image dimensions. ⁴⁴
ClamAV	Upload pipeline in Modules 2, 5, 7, 8, 9	Production: scan untrusted uploads using <code>clamd</code> ; maintain signatures with <code>freshclam</code> ; quarantine failures	Managed malware scanner	Pro: dedicated file malware scanning. Con: does not make malicious files impossible and needs signature maintenance	File scanning is defense in depth; never expose quarantined uploads. ClamAV documents both persistent <code>clamd</code> and standalone scanning models. ⁴⁵

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
Jinja2	Module 2 report HTML	Developer-controlled templates only; escape data; do not let patients provide templates	Handwritten HTML	Pro: clean template separation. Con: unsafe templates become dangerous	Template source should be trusted and version-controlled. Jinja also provides sandboxing where genuinely untrusted templates are required. ⁴⁶
WeasyPrint	Module 2 PDF finalization	HTML/CSS → PDF; bundle fonts that cover English/Devanagari; disable arbitrary remote resource use	ReportLab; headless Chromium	Pro: maintainable report layout. Con: system/native dependencies and rendering tests	Use only trusted report templates and local/allowlisted assets. ²⁷
react-pdf	Module 3 report preview	Configure PDF.js worker locally; feed authenticated blob/signed URL rather than public file URL	Browser PDF viewer	Pro: embedded React preview. Con: extra bundle/worker configuration	Prefer authenticated download or short-lived signed links for medical PDFs. ⁴⁷
i18next + react-i18next	Module 6 and visible text throughout Modules 1–9	<code>supportedLngs: ['en', 'hi'];</code> <code>fallbackLng: 'en';</code> translation namespaces per module	FormatJS	Pro: mature translation separation. Con: translation governance is still required	Never blindly translate medicine names, numeric lab values, doctor names or other identity-critical clinical strings. ²⁸

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
react-markdown	Module 7 guides/tutorial documentation	Render controlled Markdown; avoid unsafe raw HTML; sanitize if plugins broaden input	MDX	Pro: documentation stays content-driven. Con: plugin chain must be reviewed	The project documents safe defaults but warns that insecure plugins/transforms can reintroduce XSS risk. ⁴⁸
date-fns	Modules 1, 3, 4, 8, 9	Date formatting/calculations; server remains source of appointment timestamps	Day.js	Pro: modular date functions. Con: timezone/domain rules still need explicit design	Store authoritative timestamps consistently and configure hospital timezone separately. ⁴⁹
React DatePicker	Modules 8 and 9	Appointment/lab date UI; disable unavailable dates; localize UI	MUI Date Picker	Pro: React calendar component with localization/accessibility support. Con: availability still comes from server	Never determine booking availability from UI state alone. ⁵⁰
Leaflet	Modules 8 and 9	Hospital location and ambulance-request map UI	MapLibre GL	Pro: lightweight mapping. Con: tiles/geocoding are separate services	Location data can be sensitive; request it only for the explicit function. ²⁹

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
OpenStreetMap-compatible tiles/data	Modules 8 and 9	Demo mapping; configure attribution; production tile endpoint is an infrastructure decision	Commercial/self-hosted tiles	Pro: open mapping ecosystem. Con: public OSM tile servers are not a production SLA service	OSM's public tile policy imposes usage requirements; use a compliant provider or self-hosting strategy for production traffic. ³⁰
TanStack Table	Module 9 admin lists	Server-side filtering/pagination for large data; role-aware actions	AG Grid, MUI Data Grid	Pro: headless customizable tables. Con: team builds presentation layer	Do not send columns/rows an admin role is not authorized to view. ⁵¹
Recharts	Module 9 analytics	Feed aggregate metrics, not raw patient event streams	Chart.js	Pro: React-oriented charts. Con: analytics pipeline still needs definition	Prefer aggregate/de-identified operational metrics wherever individual detail is unnecessary. ⁵²
Firebase Cloud Messaging — optional	Module 4 external push	Generic notification payload; authenticated app fetches actual detail after opening	Web Push provider / none	Pro: device/browser push. Con: adds external provider dependency	Do not put diagnosis, prescriptions, lab values or other unnecessary PHI in lock-screen push text. ⁵³

Tool	Purpose and modules	Key configuration / integration	Alternatives	Pros / cons	Cost, security and privacy
Docker	Backend/AI runtime	Multi-stage image; non-root runtime; pin dependencies; health check	Podman	Pro: portable runtime. Con: image/security maintenance required	Scan/update base images and avoid baking secrets into layers. Docker's current Python guidance demonstrates containerized FastAPI deployment. ³²
Ruff	Backend development/CI	<code>ruff check</code> + <code>ruff format --check</code>	Flake8 + Black + isort	Pro: consolidated lint/format workflow. Con: configuration still required	Development tool only; pin it in dev dependencies for consistent CI. ⁵⁴
GitHub Actions	Project CI/CD	Separate frontend/backend/security/migration jobs; environment secrets; protected production deployment	GitLab CI/Jenkins	Pro: repository-integrated automation. Con: runner/minute/secrets governance	Use environment-scoped secrets and approval controls for production deployment. ⁵⁵

LLM safety contract

The LLM must be constrained to the role approved by the project documentation: gather history, identify missing history information, organize patient-provided facts, interpret OCR-derived content conservatively, and prepare a physician-readable draft. It must not turn the application into an autonomous final diagnosis or prescription system. The approved documentation states this boundary repeatedly in the core principle, AI Consultation flow, physician-control section and cross-cutting clinical-safety requirements.

filecite turn0file

Recommended internal interface:

```
from typing import Protocol
from pydantic import BaseModel
```

```

class ConsultationTurn(BaseModel):
    role: str
    text: str

class StructuredCaseDraft(BaseModel):
    chief_complaint: str | None = None
    history_of_present_illness: str | None = None
    medicines_taken: list[str] = []
    allergies: list[str] = []
    relevant_past_history: list[str] = []
    next_question: str | None = None
    ready_for_review: bool = False

class LLMProvider(Protocol):
    async def process_case(
        self,
        turns: list[ConsultationTurn],
    ) -> StructuredCaseDraft:
        ...

```

This adapter keeps the LLM vendor replaceable while Pydantic constrains the shape the application accepts. Pydantic-backed models are appropriate for typed nested validation in FastAPI. ¹⁵

Module-by-module implementation map

The following mappings follow the approved module scope in the project PDF rather than inventing additional modules. ^{filecite[turn0file0]}

Module 1 — Patient Registration

The approved registration module collects a small stable profile rather than a detailed clinical history; detailed case-taking belongs in Module 2. ^{filecite[turn0file0]}

Tool	Exact use in Module 1	Important implementation note
React + TypeScript	Registration page and form components	Model approved fields explicitly; do not expand registration into a full history form. ⁵⁶
React Hook Form	Field state, error state, submit handling	Keep accessible labels and errors. ¹¹
Zod	Client-side format/range validation	Validate email/mobile/DOB/height/weight and allowed disease values; backend repeats critical checks. ¹²

Tool	Exact use in Module 1	Important implementation note
FastAPI + Pydantic	Registration API and trusted validation	Reject malformed inputs and enforce one profile per account. ¹⁵
Supabase Auth	Account identity	<code>auth.users.id</code> should map to the application's patient profile identity. ¹⁷
Supabase PostgreSQL	<code>patient_profiles</code>	Use UUID/foreign key, timestamps and uniqueness constraints. ¹⁶
PostgreSQL RLS	Patient ownership	Patient can read/update permitted parts of own profile only. ¹⁸
i18next	English/Hindi labels	All registration labels/errors should use translation keys. ²⁸

Recommended core table:

```
create table public.patient_profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null unique references auth.users(id),
  full_name text not null,
  date_of_birth date not null,
  gender text not null,
  mobile text not null,
  email text not null,
  address text not null,
  blood_group text,
  height_cm numeric,
  weight_kg numeric,
  existing_diseases text[] not null default '{}',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Use database constraints for invariant rules rather than relying only on the React form. RLS must then restrict the profile row to its authenticated owner. Supabase recommends enabling RLS on exposed tables and separately managing grants/policies. ¹⁸

Module 2 — AI Consultation

The approved workflow is voice-led case-taking: welcome → language confirmation → patient narration → dynamic follow-up questions → optional previous-report upload → submit → AI patient report → review/edit → confirm → PDF → report history. Uploaded documents may support the history but must not silently become an autonomous diagnosis. 

Tool	Exact use in Module 2	Important implementation note
React + TypeScript	Consultation UI, microphone states, transcript, review/edit screen	Show explicit listening/processing/error states. ⁵⁶
MediaDevices <code>getUserMedia</code>	Microphone capture	Ask permission only at consultation use; microphone access requires appropriate browser security/permission handling. ⁴¹
faster-whisper	Backend STT	Configure language <code>en</code> / <code>hi</code> , model size, device and compute type. ²²
Web Speech synthesis	Speak AI prompts	Set voice language and always render same question as text. ⁴²
FastAPI	Consultation/session API	Own workflow state transitions and backend orchestration. ⁵⁷
LLM provider adapter	Dynamic question selection + structured summary	Provider TBD ; require structured output and enforce non-diagnostic prompt policy.
Pydantic	LLM output validation	Never trust arbitrary model JSON directly. ¹⁵
FastAPI <code>UploadFile</code> + <code>python-multipart</code>	Previous report uploads	Use <code>UploadFile</code> rather than loading large files entirely into memory. ⁵⁸
ClamAV	Upload malware scan	Scan before OCR/long-term storage; quarantine on failure. ⁴⁵
Supabase Storage	Original medical documents and final PDFs	Private buckets only. ¹⁹
PyMuPDF	PDF text extraction/page rendering	Try embedded text before OCRing page images. ²⁴
Pillow + OpenCV	Image decode/preprocessing	Keep original file; create derivative image for OCR. ⁵⁹
PaddleOCR	Printed/multilingual OCR and feasible handwriting processing	Select appropriate Hindi/Devanagari capability explicitly; output is unverified until reviewed. ²³
Jinja2	Physician-ready report template	Template controlled by developers. ²⁶
WeasyPrint	Final PDF generation	Test English and Devanagari fonts in container. ²⁷

Tool	Exact use in Module 2	Important implementation note
PostgreSQL	Consultation turns, extracted structured data, report versions	Keep raw/original evidence separate from normalized/AI-derived content. 16

Recommended processing pipeline:

```

Patient speaks
  ↓
Browser audio capture
  ↓
FastAPI upload/stream
  ↓
faster-whisper
  ↓
Original transcript saved
  ↓
LLM Provider Adapter
  ↓
Pydantic validation
  ↓
Next question
  ↓
Text + TTS
  ↓
Continue until sufficient history

Optional previous report
  ↓
File validation
  ↓
Malware scan
  ↓
Private temporary processing
  ↓
PyMuPDF
  ├── embedded text available → extract
  └── scanned page → OpenCV/Pillow → PaddleOCR
  ↓
Store OCR text + confidence + page references
  ↓
LLM structured extraction
  ↓
Patient report draft
  ↓
Patient Review/Edit

```

```

    ↓
Confirm
    ↓
Jinja2 + WeasyPrint
    ↓
Private final PDF
    ↓
Report History

```

Do **not** discard source evidence after AI normalization. Recommended persistence separates:

```

consultations
consultation_turns
consultation_transcripts
uploaded_documents
document_pages
ocr_extractions
structured_case_drafts
report_versions
reports

```

For every OCR-derived clinically significant item, retain enough provenance to answer questions such as “which document/page produced this item?” This is an engineering safety recommendation because the approved scope expects patients to verify/edit generated content before confirmation.

filecite turn0file0

Medical-document OCR must be treated as **assistive extraction, not ground truth**. Handwriting, unusual abbreviations, poor scans and multilingual prescriptions are exactly the cases that should trigger low-confidence display/review rather than silent acceptance.

Module 3 — My Report History

The approved patient actions are view, review, download, print, share, search and filter. Formal patient-facing delete is intentionally excluded from this module. filecite turn0file0

Tool	Exact use in Module 3	Important implementation note
React + TanStack Query	Report list/detail queries	Query only current patient's records. 60
FastAPI	Search/filter/report authorization APIs	Never accept a patient ID from the browser as sufficient authorization.
PostgreSQL	Report metadata	Index patient_id, created_at, report type/status. 16

Tool	Exact use in Module 3	Important implementation note
Supabase Storage	Final PDF objects	Keep bucket private. ¹⁹
react-pdf	In-app PDF preview	Configure worker within application build rather than relying on arbitrary third-party CDN resources. ⁶¹
Browser print	Print user-selected report	Print after authenticated report loading
Signed URLs / authenticated Storage	Download/share	Use time-limited access rather than public permanent URLs. ¹⁹
date-fns	Report date display/filter UI	Keep server query authoritative. ⁴⁹

Recommended API shape:

```
GET /api/v1/reports
GET /api/v1/reports/{report_id}
GET /api/v1/reports/{report_id}/download
POST /api/v1/reports/{report_id}/share-link
```

A share link should be deliberately short-lived and created only after authentication. Supabase signed URLs are time-limited but are not instantly invalidated merely because the user's Auth signing key later changes, so avoid unnecessarily long expiry windows. ¹⁹

Module 4 — Notification Center

Approved categories are appointment, medicine reminder, report and hospital notifications.

filecite turn0file0

Tool	Exact use in Module 4	Important implementation note
PostgreSQL	Durable notification record	Store read/unread status and category
Supabase Realtime	Update visible Notification Center	Use user-private channels/policies. ²⁰
Supabase Cron	Find due medicine/appointment reminders	One recurring due-reminder job is preferable to massive per-reminder cron proliferation. ²¹
FastAPI	Notification preference and mark-read APIs	Enforce owner authorization
FCM — optional	Out-of-app/browser push	Keep payload generic and retrieve full detail after authentication. ⁵³

Tool	Exact use in Module 4	Important implementation note
i18next / stored localized template	Patient-visible notification wording	Render according to saved language preference. ⁶²

Recommended database model:

```

notifications
  id
  patient_id
  category
  title_key
  body_key
  metadata_json
  scheduled_for
  created_at
  read_at

notification_preferences
  patient_id
  appointments_enabled
  medicine_enabled
  reports_enabled
  hospital_enabled

```

Push notifications should avoid revealing conditions, lab values, medicine names or other unnecessary clinical details on device lock screens. Prefer wording such as **“A new report is available in your account”**, with details loaded after authenticated app access.

For the initial SIH version, **Supabase Cron is preferable to Celery + Redis** because the reminder workload is database-centric and the stack already depends on PostgreSQL. If the platform later develops large asynchronous workloads, Celery can be introduced as a durable distributed task-processing layer. Celery's official documentation describes it as a distributed task queue, while Redis is a common backing infrastructure option. ⁶³

Module 5 — My Profile & Settings

The approved areas include profile information, edit profile, profile photo, account settings, privacy/permissions and notification preferences. 

Tool	Exact use in Module 5	Important implementation note
React Hook Form + Zod	Profile/settings forms	Restrict which identity/clinical fields are user-editable. ⁶⁴

Tool	Exact use in Module 5	Important implementation note
FastAPI + Pydantic	Trusted profile update API	Whitelist editable fields. ¹⁵
Supabase Auth	Account identity/session	Sensitive account changes may require re-authentication depending on final policy. ¹⁷
PostgreSQL	<code>patient_settings</code> , consent/preferences	Version consent where required. ¹⁶
Supabase Storage	Profile photo	Dedicated private or carefully scoped avatar bucket
Pillow	Validate/resize profile image	Strip unnecessary image metadata and cap dimensions. ⁶⁵
ClamAV	Uploaded profile file scan	Scan untrusted files. ⁴⁵

Recommended settings record:

```
create table public.patient_settings (
  patient_id uuid primary key references public.patient_profiles(id),
  preferred_language text not null default 'en',
  voice_language text not null default 'en',
  appointment_notifications boolean not null default true,
  medicine_reminders boolean not null default true,
  report_notifications boolean not null default true,
  hospital_notifications boolean not null default true,
  updated_at timestamptz not null default now()
);
```

Consent/permission fields that may need an audit trail should not simply be overwritten in this row. Use a separate append-oriented `consent_records` table containing the consent type, version, decision and timestamp where the final hospital/privacy design requires it.

Module 6 — Multi-Language Support

The approved initial scope is **English and Hindi**, including language selection, voice-language selection, translation, STT support and saved preference. `filecite``turn0file0`

Tool	Exact use in Module 6	Important implementation note
i18next	Translation engine	<code>supportedLngs: ['en', 'hi'], fallbackLng: 'en'.</code> ⁶²
react-i18next	React bindings	Use <code>useTranslation()</code> in module components. ⁶⁶

Tool	Exact use in Module 6	Important implementation note
JSON translation resources	UI copy	Split by module/namespace for maintainability
PostgreSQL	Language preference	Save <code>preferred_language</code> and <code>voice_language</code>
faster-whisper	English/Hindi STT	Pass known language choice instead of forcing auto-detect each turn where practical. ²²
Speech synthesis	English/Hindi voice	Set language tag and provide text fallback. ⁴²

Recommended configuration:

```
import i18n from "i18next";
import { initReactI18next } from "react-i18next";

i18n
  .use(initReactI18next)
  .init({
    supportedLngs: ["en", "hi"],
    fallbackLng: "en",
    defaultNS: "common",
    interpolation: {
      escapeValue: false,
    },
  });
```

A suitable resource organization is:

```
frontend/src/locales/
├─ en/
│   ├── common.json
│   ├── registration.json
│   ├── consultation.json
│   ├── reports.json
│   ├── notifications.json
│   ├── profile.json
│   ├── help.json
│   └── hospital.json
└─ hi/
    ├── common.json
    ├── registration.json
    └── consultation.json
```



```

├─ reports.json
├─ notifications.json
├─ profile.json
├─ help.json
└─ hospital.json

```

UI localization and clinical translation should be treated differently. Labels such as **Submit**, **My Reports**, **Appointment**, and **Help** belong in i18next. Medicine names, lab numbers, doctor names, hospital names, IDs and source-document text should not be blindly passed through UI translation. For medical narrative generated in another language, preserve the original source/transcript and treat translated text as a derived representation.

Module 7 — Help & Support

The approved scope includes user guides, contact support, FAQs, feedback, issue reporting and optional tutorial/video guidance. turn0file0

Tool	Exact use in Module 7	Important implementation note
react-markdown	Version-controlled guides	Keep raw HTML disabled unless a specific sanitized requirement exists. 
i18next	Short UI/FAQ interface text	English/Hindi support. 
FastAPI	Support/feedback/issue endpoints	Apply rate limits and validation
PostgreSQL	Tickets, feedback, FAQ metadata	Store ticket status and ownership
Supabase Storage	Issue screenshots	Private bucket and upload validation. 
ClamAV	Screenshot/attachment scanning	Scan uploaded support attachments. 
External email provider — optional/TBD	Staff email alerts	Provider deliberately unspecified

Recommended tables:

```

faq_entries
support_requests
feedback
reported_issues
tutorial_resources

```

Do not require an elderly/first-time user to leave the application and manually compose an email just to report a problem. Record the support request in the system first; external email can be a secondary staff alert.

Module 8 — Hospital Services

This module covers appointments, pharmacy, labs, departments/services, doctor directory, ambulance, hospital contact/location and an immediate emergency pathway. The documentation explicitly states that emergency access should **not** be gated behind completion of the AI consultation. [filecite turn0file0](#)

Tool	Exact use in Module 8	Important implementation note
React + TanStack Query	Service pages/data fetching	Cache relatively static directory data, refetch dynamic availability. 60
FastAPI	Hospital-service business APIs	All bookings/orders routed through backend
PostgreSQL	Doctors, schedules, slots, appointments, pharmacy, labs, ambulance requests	Use constraints for operational integrity. 16
React DatePicker	Appointment/lab date selection	Disable dates from server availability. 67
date-fns	Display/format dates	Hospital timezone configured centrally. 49
Supabase Realtime	Booking/status changes	Send only authorized patient updates. 68
Supabase Storage	Prescriptions/lab reports	Private buckets. 19
Leaflet	Hospital and ambulance location map	Separate map presentation from operational dispatch. 29
Browser Geolocation	Optional pickup coordinates	Obtain only after explicit user action/permission
OpenStreetMap-compatible tiles	Base map	Public OSM tiles suitable only within policy; production provider remains TBD. 30
i18next	English/Hindi hospital UI	Doctor/hospital proper nouns remain canonical. 62
Payment gateway	Pharmacy payment if ultimately required	TBD — not specified by approved documentation

Recommended operational tables:

```
departments
hospital_services

doctors
doctor_schedules
appointment_slots
appointments

medicines
pharmacy_inventory
pharmacy_orders
pharmacy_order_items

lab_tests
lab_slots
lab_bookings
lab_reports

ambulance_requests

hospital_configuration
```

For appointment integrity, the backend must not trust a slot shown five seconds earlier in the browser. The final database operation must atomically verify/claim the slot. A database uniqueness rule or transactional SQL/RPC should make double booking impossible even when two patients submit simultaneously.

Conceptually:

```
create unique index one_active_booking_per_slot
on appointments (appointment_slot_id)
where status in ('booked', 'confirmed');
```

Pharmacy payment processing, delivery partners, hospital HIS integration and ambulance dispatch software are **integration points**, not currently approved vendor selections. They should therefore be represented by interfaces/configuration rather than hard-coded assumptions.

Module 9 — Hospital Admin Portal

The approved design places administrative CRUD in a separate authenticated area and explicitly requires role-based authorization, separation from patient privileges and auditable administrative actions.

filecite turn0file0

Tool	Exact use in Module 9	Important implementation note
React + TypeScript	Admin application shell/ pages	May share design system but not patient authorization logic. ⁵⁶
Supabase Auth	Admin/staff authentication	Enable stronger controls/MFA for privileged accounts. ⁶⁹
FastAPI RBAC	Authoritative permission checks	Check permission on every privileged operation
PostgreSQL <code>staff_roles</code> / <code>role_permissions</code>	Current staff authorization source	Do not rely only on hidden buttons
PostgreSQL RLS	Defense-in-depth table access	Avoid relying on user-editable metadata for authorization. ¹⁸
TanStack Table	Patient/doctor/ appointment/inventory/lab tables	Prefer server pagination/filtering for growing datasets. ⁵¹
Recharts	Operational analytics	Use aggregated data where detailed patient data is unnecessary. ⁵²
Supabase Storage	Doctor images/lab reports/ admin-managed documents	Private by default
Realtime	Operational updates	Scope channels to staff role/ operation
<code>audit_logs</code>	Privileged action history	Record actor/action/resource/time/ result; never store passwords/ tokens

Recommended authorization model:

```

Supabase Auth
  ↓
Authenticated staff user
  ↓
FastAPI authorization dependency
  ↓
Query active staff role / permissions
  ↓
Permission exists?
  ├── No → 403 Forbidden
  └── Yes → perform operation

```

↓
audit_logs

For coarse UI customization, authorization metadata may be placed in protected app metadata, but Supabase specifically warns that user-editable user metadata is not an appropriate source of authorization. JWT claims can also be temporarily stale after role changes, so high-impact backend writes should validate the current staff authorization state rather than treating UI state as authority. ¹⁸

Example permissions:

```
patients.read
patients.update

doctors.read
doctors.create
doctors.update

appointments.read
appointments.manage

pharmacy.read
pharmacy.manage

labs.read
labs.manage

ambulance.read
ambulance.manage

notifications.send

analytics.read

admins.manage
```

Security, privacy, and clinical safety

The project documentation requires consent-oriented handling of sensitive health information and keeps future ABDM/HIS/EMR/ABHA/FHIR integration as an architectural direction rather than a currently implemented integration. ¹⁹filecite²⁰turn0file0²¹ The technology stack alone must therefore **not** be described as automatically “ABDM compliant”, “HIPAA compliant”, or legally certified. Institutional security, contractual, privacy, retention, data-location and regulatory review remain separate deployment responsibilities.

Row Level Security

Every exposed patient-linked table should have RLS and least-privilege grants. Supabase's current guidance explicitly recommends RLS for exposed tables and notes that grants and policies are separate controls. ¹⁸

A simplified patient policy:

```
alter table public.reports enable row level security;

revoke all on table public.reports from anon, authenticated;
grant select on table public.reports to authenticated;

create policy "patient_reads_own_reports"
on public.reports
for select
to authenticated
using (
    patient_user_id is not null
    and patient_user_id = (select auth.uid())
);
```

RLS test cases should include both **allow** and **deny** tests. Supabase now recommends database tests for RLS policies through its CLI workflow. ¹⁸

Required cases include:

Patient A can read Patient A report	PASS
Patient A can read Patient B report	FAIL
Patient A can update finalized report	FAIL
Anonymous user can read patient report	FAIL
Normal patient can call admin delete	FAIL
Authorized admin can perform allowed CRUD	PASS
Unauthorized staff can perform CRUD	FAIL

Storage

Recommended buckets:

```
patient-profile-images
previous-medical-reports
ai-patient-reports
prescriptions
lab-reports
issue-attachments
doctor-profile-images
```

Health-document buckets should be private. Downloads should use authenticated object access or short-lived signed URLs. Supabase documents signed URLs specifically for time-limited private asset access. ¹⁹

Do not use filenames containing unnecessary medical detail:

BAD:
diabetes-kidney-report-rohit-2026.pdf

BETTER:
9d4235a2-.../44c8c662-....pdf

Store human-facing filenames separately in protected metadata.

Uploads

All user uploads should follow:

Authentication

- Ownership check
- Maximum-size check
- Allowed file-extension check
- MIME/content validation
- Malware scan
- Safe parser
- Private Storage
- Processing

FastAPI's `UploadFile` is preferable for non-trivial uploads because it uses spooled file handling rather than requiring the entire upload to live in memory. ⁵⁸ ClamAV provides both a long-running scanning daemon and standalone scanning tools; use `clamd` for production throughput and keep its signature database current. ⁴⁵

Recommended initial allowed document types:

application/pdf
image/jpeg
image/png

Do not trust the browser-provided MIME header by itself.

Authentication and secrets

The frontend is permitted to contain the Supabase project URL and a publishable client key as intended by the Supabase client model, provided RLS and grants are correct. Privileged secret/service keys must remain backend-only. Supabase states that secret/service access can bypass RLS and should never be exposed to customers/browser code. ⁷⁰

For production, follow the Supabase security checklist: RLS, SSL enforcement where applicable, network restrictions for direct database connections, strong account protection/MFA and appropriate backup/recovery configuration. ⁷¹

AI/STT/OCR privacy

The preferred initial speech and OCR stack—faster-whisper + PaddleOCR—can operate within infrastructure controlled by the project, avoiding an automatic need to transmit voice and document content to a third-party speech/OCR API. faster-whisper is a CTranslate2-based Whisper implementation intended for local CPU/GPU inference. ²² PaddleOCR provides an OCR pipeline with multilingual language/model options. ²³

A future managed AI/STT/OCR provider should not be enabled merely by inserting an API key. Before sending patient data to any provider, verify at minimum:

- Data-processing terms
- Data retention behavior
- Model-training / secondary-use policy
- Data location / residency requirements
- Encryption
- Access controls
- Deletion capability
- Incident-response obligations
- Hospital approval
- Applicable legal/regulatory requirements

The same rule applies to the LLM provider. `LLM_PROVIDER` is therefore intentionally **unspecified** in this architecture.

Clinical safety

Persist the distinction between:

- PATIENT-PROVIDED FACT
- OCR-EXTRACTED TEXT
- AI-INFERRED/STRUCTURED CONTENT
- PATIENT-CORRECTED CONTENT
- CONFIRMED REPORT CONTENT

Do not collapse those concepts into one uncontrolled text blob.

Recommended report lifecycle:

```
DRAFT
↓
PATIENT_REVIEW
↓
PATIENT_EDITED (optional)
↓
CONFIRMED
↓
PDF_GENERATED
↓
FINAL_PATIENT_VERSION
```

Once confirmed, a new correction should create a version/history entry rather than silently rewriting the previously generated formal report.

The physician-control statement should also appear in the generated document, for example:

This report summarizes patient-provided history and supporting documents. It is not an autonomous medical diagnosis or prescription. Clinical assessment and treatment decisions remain with the physician.

This expresses the safety boundary already established in the approved project documentation.

filecite turn0file0

Logging

Production logs should capture operational metadata, not entire clinical conversations.

Suitable:

```
request_id
user_id/pseudonymous internal id where necessary
endpoint
HTTP status
latency
AI provider request id
OCR job id
report id
admin action
```

```
error category
timestamp
```

Avoid by default:

```
full patient transcript
prescription contents
lab values
access tokens
authorization headers
passwords
raw uploaded files
full AI prompts containing PHI
```

Admin operations should create dedicated application-level audit records because ordinary debug logs are not an adequate audit model.

Repository, environment, and configuration

A module-oriented repository structure is recommended so the code layout mirrors the approved functional architecture rather than putting every React component and FastAPI route into generic folders.

```
patient-case-taking-software/
|
|— frontend/
|   |— src/
|       |— app/
|           |— router/
|           |— providers/
|           |— layouts/
|       |— components/
|           |— ui/
|           |— accessibility/
|           |— shared/
|       |— modules/
|           |— registration/
|           |— ai-consultation/
|           |— report-history/
|           |— notifications/
|           |— profile-settings/
|           |— language/
|           |— help-support/
|           |— hospital-services/
```

```

├── admin/
├── api/
├── auth/
├── hooks/
├── locales/
│   ├── en/
│   └── hi/
├── types/
└── utils/

├── public/
├── package.json
├── package-lock.json
├── tsconfig.json
└── vite.config.ts

├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   └── v1/
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── auth.py
│   │   │   ├── permissions.py
│   │   │   └── logging.py
│   │   ├── modules/
│   │   │   ├── registration/
│   │   │   ├── consultation/
│   │   │   ├── reports/
│   │   │   ├── notifications/
│   │   │   ├── profile/
│   │   │   ├── language/
│   │   │   ├── support/
│   │   │   ├── hospital/
│   │   │   └── admin/
│   │   ├── services/
│   │   │   ├── llm/
│   │   │   ├── speech/
│   │   │   ├── ocr/
│   │   │   ├── documents/
│   │   │   ├── pdf/
│   │   │   ├── storage/
│   │   │   └── notifications/

```

```

|   |   |─ schemas/
|   |   |─ templates/
|   |   |   └─ reports/
|   |   └─ tests/
|   |
|   └─ pyproject.toml
|   └─ requirements.txt
|   └─ Dockerfile
|
└─ supabase/
    └─ config.toml
    └─ migrations/
    └─ tests/
    └─ seed.sql
|
└─ tests/
    └─ e2e/
|
└─ infra/
    └─ docker/
    └─ deployment/
|
└─ docs/
    └─ TOOLS.md
    └─ ARCHITECTURE.md
    └─ SECURITY.md
    └─ API.md
|
└─ assets/
    └─ module-1/
    └─ module-2/
    └─ module-3/
    └─ module-4/
    └─ module-5/
    └─ module-6/
    └─ module-7/
    └─ module-8/
    └─ module-9/
|
└─ .github/
    └─ workflows/
        └─ frontend-ci.yml
        └─ backend-ci.yml
        └─ database-ci.yml
        └─ deploy.yml
|
└─ .env.example
└─ .gitignore

```

```
|— docker-compose.yml
|— README.md
|— TOOLS.md
```

Supabase's current local-development workflow explicitly supports keeping `config.toml`, migrations and seed data under the repository's `supabase/` directory, making database changes reproducible instead of relying on manual Dashboard edits. ³⁸

Frontend environment variables

```
# Public frontend configuration only

VITE_APP_ENV=development
VITE_API_BASE_URL=http://localhost:8000/api/v1

VITE_SUPABASE_URL=
VITE_SUPABASE_PUBLISHABLE_KEY=

VITE_DEFAULT_LANGUAGE=en
VITE_SUPPORTED_LANGUAGES=en,hi

VITE_HOSPITAL_TIMEZONE=Asia/Kolkata

# Demo/public map endpoint.
# Production tile provider remains an infrastructure decision.
VITE_MAP_TILE_URL=
VITE_MAP_ATTRIBUTION=
```

Do **not** add this to the frontend:

```
# NEVER IN FRONTEND
SUPABASE_SECRET_KEY=
SUPABASE_SERVICE_ROLE_KEY=
LLM_API_KEY=
DATABASE_PASSWORD=
FCM_SERVER_CREDENTIALS=
```

Supabase explicitly requires privileged service/secret credentials to remain server-side because such credentials can bypass normal RLS protections. ¹⁸

Backend environment variables

```
APP_ENV=development
APP_NAME=patient-case-taking-api
```

```
APP_BASE_URL=http://localhost:8000
FRONTEND_ORIGIN=http://localhost:5173
CORS_ORIGINS=http://localhost:5173

LOG_LEVEL=INFO

# Supabase
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SECRET_KEY=

# AI provider is intentionally unspecified.
LLM_PROVIDER=
LLM_API_BASE_URL=
LLM_API_KEY=
LLM_MODEL=
LLM_TIMEOUT_SECONDS=60

# Speech-to-text
STT_PROVIDER=faster-whisper
STT_MODEL=
STT_DEVICE=cpu
STT_COMPUTE_TYPE=int8
STT_SUPPORTED_LANGUAGES=en,hi

# OCR
OCR_PROVIDER=paddleocr
OCR_LANGUAGES=en,hi
OCR_DEVICE=cpu

# Upload controls
MAX_DOCUMENT_UPLOAD_MB=20
MAX_IMAGE_PIXELS=
ALLOWED_DOCUMENT_MIME_TYPES=application/pdf,image/jpeg,image/png

# Report generation
REPORT_STORAGE_BUCKET=ai-patient-reports
REPORT_TEMPLATE=patient_report.html
REPORT_FONT_PATH=

# File scanning
MALWARE_SCANNER=clamav
CLAMAV_HOST=
CLAMAV_PORT=

# Application time
HOSPITAL_TIMEZONE=Asia/Kolkata
```

```
# Push is optional
PUSH_PROVIDER=none
FCM_PROJECT_ID=
FCM_CREDENTIALS_JSON=

# External integrations intentionally TBD
PAYMENT_PROVIDER=
HOSPITAL_HIS_PROVIDER=
ABDM_INTEGRATION_ENABLED=false
```

Use Pydantic-based settings so malformed/missing backend configuration fails early rather than appearing as a runtime surprise.

Example:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_env: str = "development"
    supabase_url: str
    supabase_publishable_key: str
    supabase_secret_key: str

    llm_provider: str
    llm_model: str

    stt_provider: str = "faster-whisper"
    ocr_provider: str = "paddleocr"

    hospital_timezone: str = "Asia/Kolkata"

    model_config = SettingsConfigDict(
        env_file=".env",
        extra="ignore",
    )

settings = Settings()
```

Suggested frontend dependencies

```
npm install \
  react-router-dom \
  @supabase/supabase-js \
```

```
@tanstack/react-query \
axios \
react-hook-form \
@hookform/resolvers \
zod \
i18next \
react-i18next \
lucide-react \
react-pdf \
react-markdown \
date-fns \
react-day-picker \
leaflet \
recharts
```

React-PDF requires appropriate PDF.js worker configuration; its documentation specifically warns that worker configuration must be set correctly in the module using the PDF components. ⁶¹

Suggested backend dependencies

```
fastapi
uvicorn[standard]
pydantic
pydantic-settings
python-multipart
supabase
httpx

faster-whisper

paddleocr
# Install the matching PaddlePaddle CPU/GPU runtime separately.

PyMuPDF
opencv-python-headless
Pillow

Jinja2
WeasyPrint

pytest
ruff
```

FastAPI requires multipart support for form/file uploads, and its official upload documentation recommends `UploadFile` for larger files because of its spooled-file model. ⁵⁸

Do not blindly install every package at `latest` during every deployment. Pin dependency versions through the project's package lockfile and Python dependency lock/pin strategy, then upgrade deliberately after automated testing.

Database migrations

Use **Supabase CLI migrations as the single source of truth** for schema/RLS changes rather than mixing untracked Dashboard changes and a separate migration system.

Typical workflow:

```
supabase start

supabase migration new add_consultation_tables

supabase db reset

supabase test db

supabase db push
```

Supabase documents migrations as version-controlled SQL changes and supports local validation before pushing them to a linked project. ⁷²

Testing, CI/CD, and deployment

Testing must focus on safety boundaries, data isolation and workflows rather than only visual component snapshots.

Testing stack

Tool	Test layer	What must be tested	Alternatives
Vitest	Frontend unit	Utilities, Zod schemas, translation helpers, reducers/helpers	Jest
React Testing Library	Frontend component	Registration validation, buttons/forms, accessibility-oriented behavior	Playwright component tests
Playwright	End-to-end	Login → register → AI flow mock → review → report; appointments; admin permissions	Cypress
pytest	Backend	APIs, services, permissions, AI schemas, OCR pipeline, booking constraints	unittest

Tool	Test layer	What must be tested	Alternatives
Supabase DB tests	Database/RLS	Owner vs non-owner access, admin permissions, constraints	pgTAP directly
Ruff	Backend static quality	Lint + formatting check	Black + Flake8 + isort
TypeScript compiler	Frontend static quality	<code>tsc --noEmit</code>	—

Minimum critical E2E scenarios:

Patient authentication

Registration

- ✓ required fields
- ✓ duplicate profile protection
- ✓ invalid date/email/mobile rejection

AI Consultation

- ✓ microphone denied
- ✓ STT unavailable
- ✓ Hindi conversation
- ✓ English conversation
- ✓ report upload
- ✓ invalid/malicious file rejection
- ✓ OCR failure
- ✓ LLM timeout
- ✓ review/edit
- ✓ confirmation
- ✓ PDF saved
- ✓ no diagnostic wording in controlled output

Report History

- ✓ own report visible
- ✓ other patient's report forbidden
- ✓ download authorization
- ✓ no patient delete action

Notifications

- ✓ preference respected
- ✓ one patient's notification not sent to another
- ✓ reminder idempotency

Hospital Services

- ✓ appointment double-book prevented
- ✓ lab slot conflict prevented

- ✓ emergency route available without AI consultation

Admin

- ✓ patient cannot access admin APIs
- ✓ role permissions enforced
- ✓ sensitive operation audited

AI evaluation

AI functionality also needs a regression dataset, because standard unit tests cannot guarantee conversational quality.

Maintain a de-identified synthetic test suite covering:

English complaints
Hindi complaints
Hindi-English mixed speech
Very short answers
Long narrative answers
Contradictory patient statements
Missing onset/duration
Medicine mentioned
Medicine response mentioned
Allergy mentioned
No prior documents
Printed report
Low-quality report
Handwritten report
OCR confidence failure
Prompt-injection text inside uploaded report
Requests for diagnosis
Requests for prescription

For each case, evaluate:

Did the AI ask an appropriate missing-history question?
Did it avoid inventing vital signs?
Did it avoid claiming a final diagnosis?
Did it preserve patient-provided facts?
Did it identify uncertain OCR content?
Did structured output validate?
Did it stop asking redundant questions?
Did the final report reflect patient corrections?

This directly tests the clinical-safety boundary defined in the approved project documentation.

filecite turn0file0

GitHub Actions

GitHub Actions supports repository-based YAML workflows and environment-specific secrets/deployment rules, making it appropriate for CI/CD. 55

A practical pipeline:

```
Pull Request
├── frontend-ci
│   ├── npm ci
│   ├── TypeScript check
│   ├── lint
│   ├── Vitest
│   └── production build
├── backend-ci
│   ├── install pinned Python deps
│   ├── ruff check
│   ├── ruff format --check
│   └── pytest
├── database-ci
│   ├── start local Supabase
│   ├── apply migrations
│   └── RLS/database tests
└── e2e
    ├── start test stack
    └── Playwright
```

Concise workflow example:

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  frontend:
```

```

runs-on: ubuntu-latest
defaults:
  run:
    working-directory: frontend
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: "22"
      cache: npm
      cache-dependency-path: frontend/package-lock.json
  - run: npm ci
  - run: npm run typecheck
  - run: npm test -- --run
  - run: npm run build

backend:
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: backend
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: pip install -r requirements.txt
    - run: ruff check .
    - run: ruff format --check .
    - run: pytest

```

The exact runtime versions should be frozen and upgraded deliberately at implementation time rather than treated as permanent architecture decisions.

Docker

FastAPI and its OCR/report dependencies should be containerized because PaddleOCR, faster-whisper and WeasyPrint introduce runtime/system dependencies that are easier to manage in a reproducible image. Docker's official Python guidance includes containerizing FastAPI and recommends separating build/runtime concerns. ³²

Conceptual Dockerfile:

```

FROM python:3.12-slim AS runtime

WORKDIR /app

```

```

# Install only the native libraries actually required by
# WeasyPrint/PaddleOCR/OpenCV/ClamAV integration in your build.

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

RUN useradd --create-home appuser
USER appuser

EXPOSE 8000

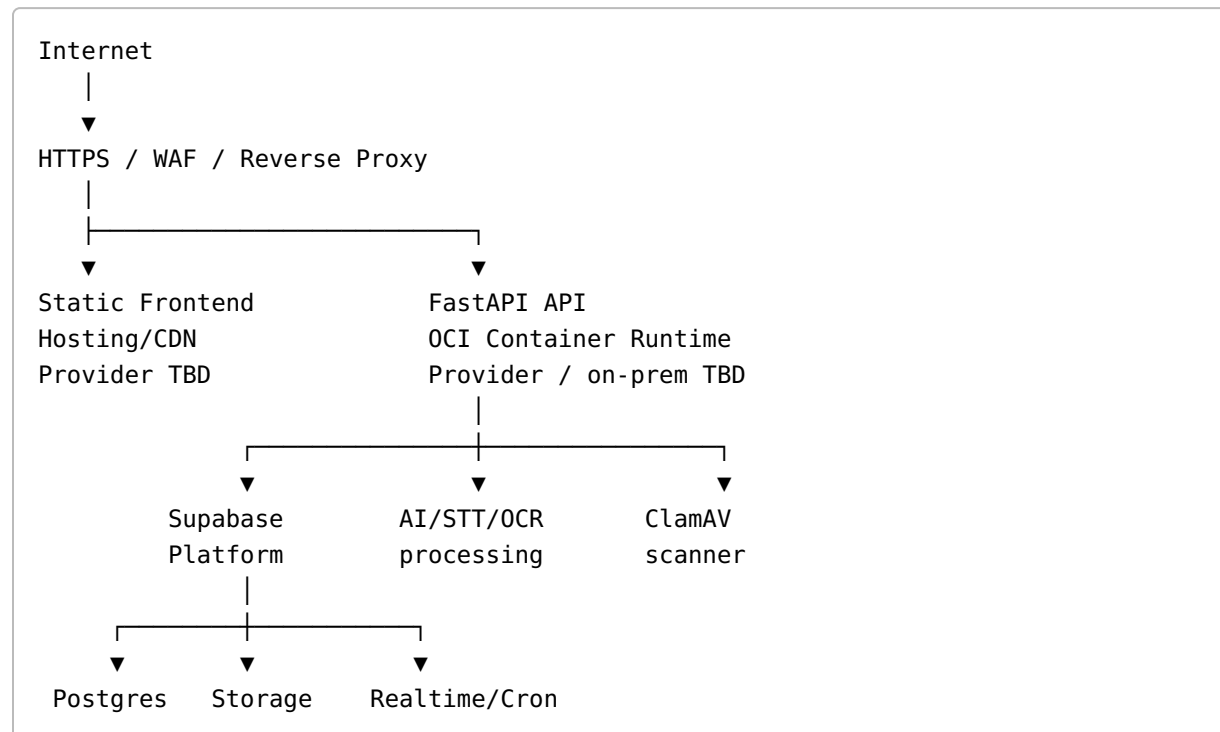
CMD ["uvicorn", "app.main:app", "--host=0.0.0.0", "--port=8000"]

```

Do not put `.env`, model-provider keys or Supabase secret keys in Docker image layers.

Deployment recommendation

No cloud provider is mandated by the functional design, so infrastructure should remain provider-neutral.



For an SIH prototype, Supabase's hosted platform is practical because the project uses its PostgreSQL, Auth, Storage and Realtime services as one integrated data layer. Production pricing is usage/tier dependent and currently varies with compute, database size, storage, egress, active users and platform features, so the

team should verify current pricing during capacity planning rather than encode dollar assumptions in the architecture. ⁷⁵

For hospital production, choose the actual frontend host, container runtime, AI hardware and network topology only after confirming institutional requirements for health-data handling, backups, recovery, availability, network controls and location/residency. Supabase's own production checklist calls out RLS, SSL enforcement, network restrictions, MFA and backup/recovery planning. ⁷¹

The local Supabase development stack must not be exposed as production infrastructure; Supabase states that its local stack is designed for development and is not production hardened. ⁷⁶

Production environments

Use separate projects/environments:

LOCAL
developer-only synthetic data
DEVELOPMENT
integration/testing data only
STAGING
production-like configuration
synthetic/de-identified test data preferred
PRODUCTION
real patient data
strongest controls

Never share production Supabase credentials with local development.

Migration notes and final tools matrix

The architecture deliberately avoids making most module code dependent on a particular third-party vendor. The main migration boundary is a service/interface layer around external capabilities.

Migration and replacement strategy

Current choice	Migration target	Migration method	Main caveat
React	Vue/Angular	Rewrite UI while retaining FastAPI contracts	Largest frontend migration
Vite	Another React build framework	Preserve module components where compatible	Environment/routing conventions change

Current choice	Migration target	Migration method	Main caveat
React Hook Form + Zod	Formik/other schemas	Replace form layer incrementally	Validation behavior must remain equivalent
Axios	Native <code>fetch</code>	Replace shared API client	Rebuild interceptor/error conventions
FastAPI	Django/Node/.NET	Preserve OpenAPI/API behavior and DB schema	AI/document Python services may remain separate
Supabase PostgreSQL	Standard managed/self-hosted PostgreSQL	Export/apply SQL schema and data	Replace Supabase-specific Auth/RLS helper assumptions
Supabase Auth	Keycloak/other IdP	Map external subject IDs to patient/staff identities	RLS/JWT claims need redesign
Supabase Storage	S3-compatible object store	Migrate objects + metadata paths	Recreate signed URL/access model
Supabase Realtime	WebSockets/Redis	Replace event transport behind notification service	Channel authorization must be recreated
Supabase Cron	Celery/APScheduler/managed scheduler	Keep due-reminder query/service interface	Idempotency must be maintained
faster-whisper	Managed STT	Implement <code>SpeechToTextProvider</code>	Privacy/cost/data transfer changes
PaddleOCR	Managed Document AI/Tesseract	Implement <code>OCRProvider</code>	Output schemas/confidence differ
LLM provider TBD	Another LLM	Implement same <code>LLMProvider</code> contract	Re-run safety/evaluation regression suite
Browser Speech Synthesis	Managed/local TTS	Implement <code>TextToSpeechProvider</code>	Browser vs server audio workflow changes
WeasyPrint	ReportLab/Chromium	Keep normalized report data + template contract	Layout can change
Leaflet	MapLibre/other map SDK	Replace presentation adapter	Tile/provider terms differ

Current choice	Migration target	Migration method	Main caveat
Public OSM-compatible demo tiles	Paid/self-hosted tiles	Change configured tile endpoint	Attribution/licensing/policy still apply
FCM	Another push provider	Replace push adapter	Token lifecycle differs

The most important migration rule is: **never let vendor-specific payloads become your canonical medical data model.**

Prefer:

```
PatientReport
ConsultationTurn
OCRExtraction
Notification
Appointment
LabBooking
```

over:

```
SomeVendorLLMResponse
SomeVendorOCRDocument
SomeVendorNotificationPayload
```

That separation is what makes migration practical.

Final tools matrix

Tool / technology	Category	Modules	Primary function	Status
TypeScript	Frontend language	1-9	Typed frontend code	Required 5
React	Frontend	1-9	Patient/admin UI	Required 6
Vite	Build	1-9	Dev server/build	Required 7
React Router	Routing	1-9	Page/module routes	Required 8
Tailwind CSS	Styling	1-9	Responsive styling	Required 9

Tool / technology	Category	Modules	Primary function	Status
shadcn/ui	UI	1-9	Reusable UI primitives	Recommended ¹⁰
Lucide React	Icons	1-9	UI icon system	Recommended
React Hook Form	Forms	1, 5, 7, 8, 9	Form state	Required ¹¹
Zod	Validation	1, 5, 7, 8, 9	Client schemas	Required ¹²
Axios	HTTP	1-9	FastAPI requests	Recommended ¹⁴
TanStack Query	Server-state	1-9	Queries/cache/mutations	Required ⁶⁰
@supabase/supabase-js	Auth/Realtime client	1-9	Auth sessions + realtime	Required ³⁹
Python	Backend language	1-9	APIs/AI/OCR	Required
FastAPI	Backend	1-9	REST/business services	Required ⁵⁷
Uvicorn	ASGI server	1-9	Run FastAPI	Required
Pydantic	Backend validation	1-9	API/AI schemas	Required ¹⁵
pydantic-settings	Configuration	1-9	Backend env config	Recommended
supabase-py	Backend Supabase SDK	1-9	Data/Storage integration	Required ⁴⁰
PostgreSQL	Database	1-9	System of record	Required ¹⁶
Supabase Auth	Identity	1, 5, 9 + protected app	Authentication	Required ¹⁷
PostgreSQL RLS	Authorization	1-9	Patient data isolation	Required ¹⁸
Supabase Storage	Files	2, 3, 5, 7, 8, 9	Medical/private files	Required ¹⁹
Supabase Realtime	Realtime	4, 8, 9	Live notifications/status	Required ⁶⁸
Supabase Cron / pg_cron	Scheduling	4	Due reminder processing	Required for scheduled reminders ²¹

Tool / technology	Category	Modules	Primary function	Status
Supabase CLI	DB DevOps	1-9	Migrations/local DB/RLS tests	Required ⁷⁷
MediaDevices/ getUserMedia	Browser audio	2	Microphone capture	Required ⁴¹
faster-whisper	Speech-to-text	2, 6	English/Hindi transcription	Required baseline ²²
Web Speech synthesis	TTS	2, 6	AI voice playback	Prototype primary ⁴²
LLM provider adapter	AI	2	Dynamic questions/report structuring	Required; vendor TBD
HTTPX/provider SDK	AI transport	2	Backend external model call	Depends on provider
<code>python-multipart</code>	File upload	2, 5, 7, 8, 9	Multipart uploads	Required ⁵⁸
ClamAV	Upload security	2, 5, 7, 8, 9	Malware scanning	Production recommended ⁴⁵
PaddleOCR	OCR	2	Medical-document OCR	Required baseline ⁷⁸
PaddlePaddle	ML runtime	2	PaddleOCR execution	Required with PaddleOCR
PyMuPDF	PDF processing	2	Extract/render PDFs	Required ²⁴
OpenCV headless	Image processing	2	OCR preprocessing	Recommended ⁴³
Pillow	Image processing	2, 5, 7, 9	Decode/resize images	Recommended ⁶⁵
Jinja2	Reporting	2	HTML report templates	Required ²⁶
WeasyPrint	PDF	2	Final PDF generation	Required ²⁷
react-pdf	Report viewer	3	In-app PDF display	Recommended ⁶¹
Browser Print API	Reports	3	Print report	Required feature

Tool / technology	Category	Modules	Primary function	Status
Signed Storage URLs	Reports/files	3, 8	Controlled temporary sharing	Required where sharing is enabled ¹⁹
i18next	Localization	1–9 / primary in 6	Translation engine	Required ⁶²
react-i18next	Localization	1–9 / primary in 6	React translation bindings	Required ⁶⁶
react-markdown	Help	7	User-guide rendering	Recommended ⁴⁸
date-fns	Date handling	1, 3, 4, 8, 9	Dates/times	Recommended ⁴⁹
React DayPicker	Scheduling UI	8, 9	Appointment/lab calendar	Recommended ⁶⁷
Leaflet	Maps	8, 9	Hospital/ambulance maps	Required if maps shown ²⁹
OpenStreetMap-compatible source	Maps	8, 9	Base map	Demo option; production provider TBD ³⁰
Browser Geolocation API	Location	8	Pickup/current location	Optional
FCM	Push	4	External push alerts	Optional ⁵³
TanStack Table	Admin UI	9	Data tables	Recommended ⁷⁹
Recharts	Admin analytics	9	Charts	Recommended ⁸⁰
Audit-log service/table	Security	9	Admin audit trail	Required
Vitest	Testing	1–9	Frontend unit tests	Required ⁷³
React Testing Library	Testing	1–9	Component/user interaction tests	Required ⁷⁴
Playwright	Testing	1–9	Cross-browser E2E	Required ⁸¹
pytest	Testing	1–9	Backend tests	Required

Tool / technology	Category	Modules	Primary function	Status
Supabase DB tests	Security testing	1-9	RLS/DB constraints	Required ¹⁸
Ruff	Python quality	1-9 backend	Lint/format	Recommended ⁸²
Docker	Infrastructure	Complete project	Backend/AI packaging	Required ³²
Git	Version control	Complete project	Code history	Required
GitHub	Repository	Complete project	Source collaboration	Recommended
GitHub Actions	CI/CD	Complete project	Automated test/build/deploy	Required ³¹
Frontend static host	Deployment	Complete project	Host compiled SPA	Provider TBD
OCI/container host	Deployment	Complete project	Host FastAPI	Provider/on-prem TBD
Supabase Platform	Data hosting	Complete project	Postgres/Auth/Storage/Realtime	Primary prototype/initial option ⁷⁵
LLM infrastructure	AI hosting	2	Model inference	Provider TBD
GPU compute	AI/OCR/STT	2	Optional acceleration	Provider/on-prem TBD
Payment gateway	Pharmacy	8	Online payment if retained in scope	TBD
HIS/EMR connector	Integration	8, 9 / future	Hospital-system interoperability	TBD
ABDM/ABHA/FHIR connector	Integration	Future cross-cutting	Interoperability direction	TBD; not currently implemented filecite0turn0file0

The final implementation baseline is therefore:

```

FRONTEND
React
+ TypeScript
+ Vite
+ React Router
+ Tailwind CSS

```

- + shadcn/ui
- + React Hook Form
- + Zod
- + TanStack Query
- + Axios
- + i18next/react-i18next

BACKEND

Python

- + FastAPI
- + Pydantic
- + Uvicorn
- + supabase-py

DATABASE / PLATFORM

Supabase PostgreSQL

- + Supabase Auth
- + Row Level Security
- + Supabase Storage
- + Supabase Realtime
- + Supabase Cron
- + Supabase CLI migrations

AI CONSULTATION

Browser MediaDevices

- + faster-whisper
- + provider-neutral LLM adapter
- + Pydantic structured output
- + browser TTS initially

DOCUMENT INTELLIGENCE

FastAPI UploadFile

- + ClamAV
- + PyMuPDF
- + Pillow
- + OpenCV
- + PaddleOCR

REPORTING

Jinja2

- + WeasyPrint
- + Supabase private Storage
- + react-pdf

MULTI-LANGUAGE

i18next

- + react-i18next
- + English

- + Hindi
- + language-aware STT/TTS

HOSPITAL SERVICES

- React DatePicker
- + date-fns
- + Leaflet
- + OpenStreetMap-compatible map source
- + PostgreSQL booking constraints

ADMIN

- Supabase Auth
- + FastAPI RBAC
- + RLS
- + TanStack Table
- + Recharts
- + audit_logs

TESTING

- Vitest
- + React Testing Library
- + Playwright
- + pytest
- + Supabase DB/RLS tests
- + Ruff

DEVOPS

- Git
- + GitHub
- + Docker
- + GitHub Actions
- + provider-neutral deployment

This tool selection implements the approved nine-module Patient Case-Taking Software while preserving its central architectural boundary: **AI assists with history acquisition, document digitization and structured reporting; the physician remains responsible for diagnosis, examination, treatment and prescription.** turn0file0

   <https://react.dev/>
<https://react.dev/>

   <https://fastapi.tiangolo.com/ja/>
<https://fastapi.tiangolo.com/ja/>

 <https://supabase.com/docs>
<https://supabase.com/docs>

4 19 <https://supabase.com/docs/guides/storage/serving/downloads>
<https://supabase.com/docs/guides/storage/serving/downloads>

5 <https://www.typescriptlang.org/>
<https://www.typescriptlang.org/>

7 <https://vite.dev/guide/>
<https://vite.dev/guide/>

8 <https://reactrouter.com/dom/home>
<https://reactrouter.com/dom/home>

9 <https://tailwindcss.com/docs/installation/tailwind-cli>
<https://tailwindcss.com/docs/installation/tailwind-cli>

10 <https://ui.shadcn.com/docs>
<https://ui.shadcn.com/docs>

11 64 <https://www.react-hook-form.com/>
<https://www.react-hook-form.com/>

12 <https://zod.dev/>
<https://zod.dev/>

13 60 <https://tanstack.com/query/latest/docs/framework/react>
<https://tanstack.com/query/latest/docs/framework/react>

14 <https://axios-http.com/ptBR/docs/intro>
<https://axios-http.com/ptBR/docs/intro>

15 <https://fastapi.tiangolo.com/tutorial/body-nested-models/>
<https://fastapi.tiangolo.com/tutorial/body-nested-models/>

16 36 <https://supabase.com/docs/guides/database/overview>
<https://supabase.com/docs/guides/database/overview>

17 37 <https://supabase.com/docs/guides/auth>
<https://supabase.com/docs/guides/auth>

18 70 <https://supabase.com/docs/guides/database/postgres/row-level-security>
<https://supabase.com/docs/guides/database/postgres/row-level-security>

20 68 <https://supabase.com/docs/guides/realtime>
<https://supabase.com/docs/guides/realtime>

21 <https://supabase.com/docs/guides/cron>
<https://supabase.com/docs/guides/cron>

22 <https://github.com/SYSTRAN/faster-whisper>
<https://github.com/SYSTRAN/faster-whisper>

23 78 <https://paddlepaddle.github.io/PaddleOCR/v3.0.0/en/index.html>
<https://paddlepaddle.github.io/PaddleOCR/v3.0.0/en/index.html>

24 <https://pymupdf.readthedocs.io/en/latest/>
<https://pymupdf.readthedocs.io/en/latest/>

25 43 https://docs.opencv.org/4.5.2/d6/d00/tutorial_py_root.html
https://docs.opencv.org/4.5.2/d6/d00/tutorial_py_root.html

26 46 <https://jinja.palletsprojects.com/en/stable/templates/>
<https://jinja.palletsprojects.com/en/stable/templates/>

27 <https://doc.courtbouillon.org/weasyprint/stable/>
<https://doc.courtbouillon.org/weasyprint/stable/>

28 62 <https://www.i18next.com/overview/configuration-options>
<https://www.i18next.com/overview/configuration-options>

29 <https://leafletjs.com/reference>
<https://leafletjs.com/reference>

30 <https://operations.osmfoundation.org/policies/tiles/>
<https://operations.osmfoundation.org/policies/tiles/>

31 55 <https://docs.github.com/en/actions>
<https://docs.github.com/en/actions>

32 <https://docs.docker.com/guides/python/>
<https://docs.docker.com/guides/python/>

33 39 <https://supabase.com/docs/reference/javascript/installing>
<https://supabase.com/docs/reference/javascript/installing>

34 <https://supabase.com/docs/guides/auth/jwts>
<https://supabase.com/docs/guides/auth/jwts>

38 72 77 <https://supabase.com/docs/guides/local-development/database-migrations>
<https://supabase.com/docs/guides/local-development/database-migrations>

40 <https://supabase.com/docs/reference/python/installing>
<https://supabase.com/docs/reference/python/installing>

41 <https://www.w3.org/TR/mediacapture-streams/>
<https://www.w3.org/TR/mediacapture-streams/>

42 <https://webaudio.github.io/web-speech-api/>
<https://webaudio.github.io/web-speech-api/>

44 59 65 <https://pillow.readthedocs.io/en/stable/>
<https://pillow.readthedocs.io/en/stable/>

45 <https://docs.clamav.net/manual/Usage.html>
<https://docs.clamav.net/manual/Usage.html>

47 61 <https://github.com/wojtekmaj/react-pdf>
<https://github.com/wojtekmaj/react-pdf>

48 <https://github.com/remarkjs/react-markdown>
<https://github.com/remarkjs/react-markdown>

49 <https://date-fns.org/>
<https://date-fns.org/>

50 67 <https://daypicker.dev/>

<https://daypicker.dev/>

51 79 <https://tanstack.com/table/latest/docs/framework/react>

<https://tanstack.com/table/latest/docs/framework/react>

52 80 <https://recharts.github.io/en-US/guide/>

<https://recharts.github.io/en-US/guide/>

53 <https://firebase.google.com/docs/cloud-messaging/web/get-started>

<https://firebase.google.com/docs/cloud-messaging/web/get-started>

54 82 <https://docs.astral.sh/ruff/>

<https://docs.astral.sh/ruff/>

58 <https://fastapi.tiangolo.com/tutorial/request-files/>

<https://fastapi.tiangolo.com/tutorial/request-files/>

63 <https://docs.celeryq.dev/en/stable/>

<https://docs.celeryq.dev/en/stable/>

66 <https://react.i18next.com/latest/usetranslation-hook>

<https://react.i18next.com/latest/usetranslation-hook>

69 71 <https://supabase.com/docs/guides/deployment/going-into-prod>

<https://supabase.com/docs/guides/deployment/going-into-prod>

73 <https://vitest.dev/>

<https://vitest.dev/>

74 <https://testing-library.com/docs/react-testing-library/intro/>

<https://testing-library.com/docs/react-testing-library/intro/>

75 <https://supabase.com/pricing>

<https://supabase.com/pricing>

76 <https://supabase.com/docs/guides/local-development/cli-workflows>

<https://supabase.com/docs/guides/local-development/cli-workflows>

81 <https://playwright.dev/>

<https://playwright.dev/>